

Requirements Analysis Document (v2.0)

1. Stakeholders and User Classes

Understanding the participants is the first step in defining the system's "Understanding the Problem Domain."

User Classes

- **Individual Developers (Primary):** Users who interact with the dashboard for real-time code analysis, refactoring suggestions, and style compliance.
- **Team Leads (Favored Users):** Users requiring high-security encryption for proprietary code and advanced reporting to track technical debt across multiple submissions.
- **System Administrators:** Responsible for managing the Supabase/PostgreSQL database, user authentication policies, and monitoring API health.

Stakeholder Roles

- **Product Visionary:** Defines business goals and the "Databricks-inspired" scope.
- **Technical Lead:** Oversees the Python-based backend, AI pattern recognition feasibility, and system integration.

2. User Requirements & Use Cases

User goals focus on the "What" of the system—the tasks users perform to gain value.

Use Cases (Interaction Models)

- **UC-1: Authenticate User:** Securely log in to access personal history and encrypted data.
- **UC-2: Submit Code for Analysis:** Paste source code into the dashboard to trigger the analysis engine.
- **UC-3: View Quality Feedback:** Review bugs, style violations (PEP 8), and refactoring suggestions.
- **UC-4: Retrieve Submission History:** View and compare past analysis results stored in the database.
- **UC-5: Visualize Dataflow (Stretch):** View an interactive flowchart of program dependencies.

3. Functional Requirements

These define the specific behaviors the system must exhibit.

- **REQ-1:** The system shall provide a JavaScript-based web dashboard for users to paste and submit source code.

- **REQ-2:** The system shall encrypt all user-submitted source code using industry-standard encryption before storage in the PostgreSQL database.
- **REQ-3:** The system shall implement Row Level Security (RLS) to ensure users can only access their own submissions.
- **REQ-4:** The system shall use a static analysis engine to detect deviations from the PEP 8 coding standard.
- **REQ-5:** The system shall utilize an AI-assisted model to identify risky code patterns, such as duplicated logic or improper abstractions.
- **REQ-6:** The system shall generate a structured "Quality Report" summarizing all detected bugs and style issues.
- **REQ-7 (Stretch):** The system shall generate a graphical representation of dataflow paths using an intermediate representation of the code.

4. Non-Functional Requirements (Quality Attributes)

These define the constraints and quality targets of the system.

Performance & Usability

- **REQ-8 (Efficiency):** The system shall produce and display analysis results within 5 seconds of the user's submission.
- **REQ-9 (Learnability):** A new user shall be able to complete their first full code analysis within 2 minutes of their initial login.
- **REQ-10 (Responsiveness):** The dashboard must remain interactive and display a loading state while backend analysis is in progress.

Security & Reliability

- **REQ-11 (Data Integrity):** The system shall handle codebases of up to 5,000 lines without performance degradation or crashes.
- **REQ-12 (Confidentiality):** User authentication must be required for all database read/write operations.

5. Technical Constraints & Environment

These are the "How" constraints imposed by the architectural design.

- **Frontend Architecture:** JavaScript-based web application deployed via **Vercel**.
- **Backend Architecture: FastAPI (Python)** orchestrating the static analysis and AI prediction modules.
- **Database: Supabase (PostgreSQL)** for authentication, metadata, and encrypted persistence.

- **Dependency Management:** **Docker** for local environment consistency and **Poetry** for Python package management.
- **Analysis Libraries:** `tree-sitter` (for polyglot parsing), `flake8` (style), and `ast` (structural mapping).